Trees at Work: Ordered Sets and the BST/Sorting Connection

Programming and Data Structures — Week 7, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to implement range queries and floor/ceiling queries on a binary search tree, using its ordering property directly rather than a linear scan; explain treesort as a genuine sorting algorithm derived directly from BST insertion and inorder traversal, and explain precisely why it is rarely used in practice despite being correct; explain how a balanced BST provides an ordered-set abstraction that a hash table, covered next week, fundamentally cannot offer; and choose between an array, a hash table, and a balanced BST for a concretely described real workload, justifying the choice.

Outline

Today covers a library-shelf-with-labels analogy for ordered queries; range queries, retrieving every value between two bounds; floor and ceiling queries, finding the nearest value below or above a target; treesort, a genuine sorting algorithm that falls directly out of BST insertion and inorder traversal; why treesort is rarely used in practice, examined precisely; the ordered-set abstraction a balanced BST provides, and why it matters as a point of contrast heading into next week's hash tables; the MTech-track material on augmented trees supporting order statistics; an in-class quiz; and a summary.

The library-shelf-with-labels analogy

A library organized by call number can answer "show me everything between call numbers 500 and 600" efficiently, by locating the start of that range and walking shelves contiguously until exceeding it. An arbitrarily ordered pile of books cannot answer this without checking every single book. A balanced BST provides exactly this same "efficiently walk a contiguous range" capability over its stored values, in $O(\log n + k)$ time for k matching results — locating the range's start costs $O(\log n)$, exactly as search does, and then k additional steps retrieve the results themselves.

Range query, in code

```python
def range_query(node, low, high, result=None):
    if result is None:
        result = []
    if node is None:
        return result
    if node.value > low:
        range_query(node.left, low, high, result)   # left subtree may contain values in rang
    if low <= node.value <= high:
        result.append(node.value)
    if node.value < high:
        range_query(node.right, low, high, result)  # right subtree may contain values in ra
    return result
```

This range query uses the BST property to prune entire subtrees that cannot possibly contain a value within `[low, high]`. If the current node's value is less than or equal to `low`, the entire left subtree is guaranteed to hold only values that are also too small, since the BST property guarantees they are all less than the current node — that whole subtree is skipped without ever being visited. The symmetric pruning applies to the right subtree when the current node's value already exceeds `high`. This selective pruning is precisely what keeps the cost proportional to $\log n$ plus the number of actual results, rather than the tree's total size.

Tracing range_query(3, 9) on Lecture 1's example tree

```
Tree:        8
           /   \
          3     10
         / \      \
        1   6      14

range_query(8, 3, 9):
  8>3 → recurse left into 3
    3>3? no → skip further left recursion into node 1 entirely (1 is too small)
    3 in [3,9]? yes → append 3
    3<9 → recurse right into 6
      6>3 → recurse left (None, nothing)
      6 in [3,9]? yes → append 6
      6<9 → recurse right (None, nothing)
  8 in [3,9]? yes → append 8
  8<9 → recurse right into 10
```

```
    10>3 → recurse left (None, nothing)
    10 in [3,9]? no → don't append
    10<9? no → SKIP recursing into 14 entirely (14 is too large)
Result: [3, 6, 8]
```

Tracing the range query for values between 3 and 9 inclusive: the search correctly skips recursing into node 1 (since `3 > 3` is false, meaning node 3 itself is the lower boundary, and nothing smaller needs checking) and correctly skips recursing into node 14 entirely, since node 10's value, 10, already exceeds the upper bound 9 — the entire subtree rooted at 10's right child cannot contain anything in range. The correct result, `[3, 6, 8]`, is produced while never visiting nodes 1 or 14 at all.

Tracing floor(target=7) on Lecture 1's example tree

```
Tree:        8
           /   \
          3     10
         / \      \
        1   6      14

floor(8, 7):
  8 != 7, 8 > 7 → recurse left into node 3
    floor(3, 7):
      3 != 7, 3 < 7 → check right subtree first
        floor(6, 7):
          6 != 7, 6 < 7 → recurse right (None) → right_floor = None
          right_floor is None → return node's OWN value, 6
      right_floor = 6, not None → return 6
Result: floor(7) = 6
```

Tracing `floor(root, 7)` completely: at the root, 8, since $8 > 7$, the search moves left into node 3, since only the left subtree can possibly hold a value at or below 7. At node 3, since $3 < 7$, the search checks the right subtree first (which might hold a larger valid floor closer to 7) before falling back to node 3's own value. At node 6, since $6 < 7$ and its right child is `None`, the recursive floor search returns `None`, causing node 6 to return its own value, 6, as the floor within its own subtree. This value, 6, propagates back up as the correct final answer — the largest value in the entire tree at or below 7.

Floor and ceiling queries

```python
def floor(node, target):
    if node is None:
        return None
    if node.value == target:
        return node.value
    if node.value > target:
        return floor(node.left, target)      # floor must be smaller - only left subtree can
    right_floor = floor(node.right, target)
    return right_floor if right_floor is not None else node.value
```

Floor finds the largest stored value that is less than or equal to a target — useful, for instance, in a pricing system finding the nearest-lower valid price tier. Ceiling is its exact symmetric counterpart, finding the smallest stored value greater than or equal to the target. Both queries follow a single root-to-somewhere path through the tree, exactly like search, costing $O(h)$ — neither requires examining the tree's full contents, unlike a linear scan over an unordered structure, which would need $O(n)$ for the same query.

Treesort: a sorting algorithm, for free

```python
def treesort(arr):
    root = None
    for value in arr:
        root = bst_insert(root, value)    # Week 7 Lecture 2's insert
    return inorder(root)                    # Week 7 Lecture 1's inorder traversal
```

Treesort inserts every element of an input array into a binary search tree, one at a time, then reads the fully sorted result back out via inorder traversal — both operations already fully built earlier this week. This is a genuine, correct sorting algorithm, not merely an analogy, arising directly from combining Lecture 1's traversal with Lecture 2's insertion. If the tree is kept balanced using Lecture 3's AVL insertion, each of the n insertions costs $O(\log n)$, giving $O(n \log n)$ total — matching Week 6's merge sort and quicksort exactly, via an entirely different mechanism.

Tracing treesort on `[5, 2, 8, 1, 9]`

```
insert 5: root=5
insert 2: 5→left: 2
insert 8: 5→right: 8
insert 1: 5→left:2→left:1
```

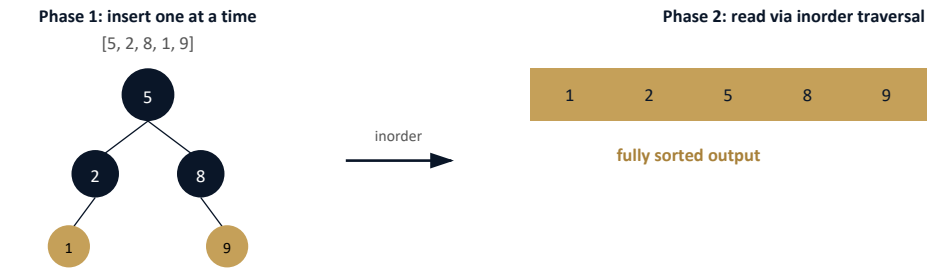
```
insert 9: 5→right:8→right:9

Tree:        5
            / \
           2   8
          /     \
         1       9

inorder(5): [1, 2, 5, 8, 9]
```

Tracing treesort end to end on `[5, 2, 8, 1, 9]`: each value is inserted in turn using Lecture 2's `bst_insert`, building the tree shown. Reading this tree via Lecture 1's inorder traversal (left, root, right at every node) produces `[1, 2, 5, 8, 9]` — the fully sorted result, obtained without ever writing a comparison-based sorting loop directly; the sorting work was entirely absorbed into where each value happened to land during insertion.

Why treesort is rarely used, precisely

Treesort is rarely used in practice for two concrete reasons. First, using a plain, unbalanced BST, an already-sorted input degenerates exactly to Lecture 1's worst-case chain, giving $O(n^2)$ — precisely the failure mode this entire week has been building toward avoiding. Second, even using an AVL tree to guarantee $O(n \log n)$, the actual constant factor is significantly worse than merge sort's: every single insertion allocates a new tree node and follows pointer references, overhead that merge sort's simple array indexing and contiguous-memory merging entirely avoids. Memory use is also worse in practice — each of the n tree nodes needs its own `left` and `right` pointer storage, in addition to its value, a real constant-factor cost beyond merge sort's already- acknowledged O(n) extra array space.

The picture: treesort's two phases



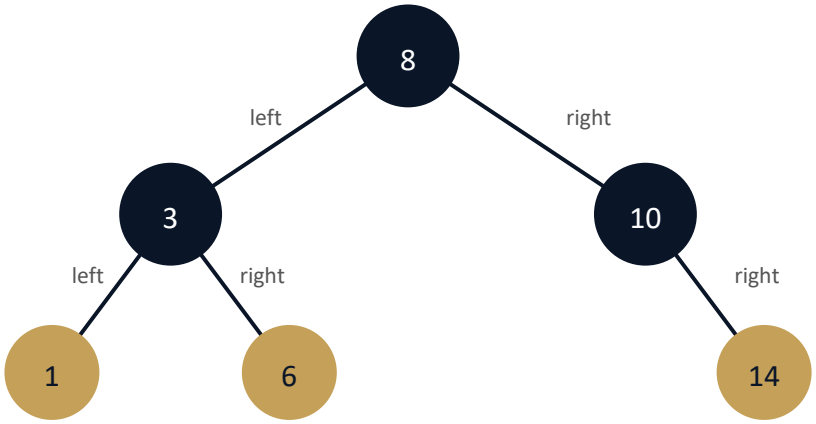
The sorting work is entirely absorbed into insertion order; reading the result is a single, simple traversal.

Treesort's two phases — build the tree via repeated insertion, then read it back via inorder traversal — are each individually simple, and their combination is genuinely correct, even though the resulting algorithm is rarely the practical choice compared to Week 6's dedicated sorting algorithms.

The ordered-set abstraction, and why it matters ahead of Week 8

A balanced binary search tree maintains its elements in sorted order continuously, supporting insertion, deletion, search, range queries, and floor/ceiling queries, all in $O(\log n)$ — this is precisely the "maintained ranking structure" that Week 6 Lecture 4's leaderboard case study identified as needed but did not yet build; a balanced BST is exactly such a structure. Next week's hash tables offer an even faster $O(1)$ average-case cost for search, insert, and delete — but, critically, cannot efficiently support range, floor, or ceiling queries at all, since a hash table maintains no ordering among its keys whatsoever. This trade-off — order-preserving but $O(\log n)$, versus order-destroying but $O(1)$ average — is the central design decision opening next week.

The picture: range query pruning, visualized



Leaves (gold): nodes with no children. Root at top, branching downward via left/right references.

Reusing this week's running example tree one final time: a range query for values between 3 and 9 visits nodes 3, 6, 8, and 10, while never descending into the subtrees containing 1 or 14 at all — the BST property structurally guarantees those subtrees are irrelevant to this particular query, without needing to check each of their contents individually.

MTech addition — augmented trees: order statistics

```
class AugmentedNode(TreeNode):
    def __init__(self, value):
        super().__init__(value)
        self.subtree_size = 1     # count of nodes in this node's own subtree, including itse

def select_kth(node, k):
    left_size = node.left.subtree_size if node.left else 0
    if k == left_size:
        return node.value
    elif k < left_size:
        return select_kth(node.left, k)
    else:
        return select_kth(node.right, k - left_size - 1)
```

Augmenting each tree node with a `subtree_size` field, maintained incrementally during insertion and deletion, enables answering "find the k-th smallest element" in $O(\log n)$: at each node, comparing k against the size of the left subtree determines whether the answer lies in the left subtree, is the current node itself, or lies in the right subtree (adjusted for how many elements have already been passed over). Without this augmentation, the same query would require a full $O(n)$ inorder traversal, counting elements as they are visited. This illustrates a general and powerful technique in data structure design: storing extra, incrementally-maintained metadata at each node to support an entirely new class of query efficiently, without changing the tree's core structure.

Comparing every structure covered so far, for one final table

This table places every major structure covered across the course so far side by side on four key properties. Arrays and balanced BSTs both maintain order and support range queries, but arrays pay O(n) for insertion while BSTs pay only O(log n). Linked lists offer fast front insertion but no ordering benefit for search or range queries. Hash tables, previewed here ahead of next week's full treatment, offer the fastest average-case search and insert, but at the cost of abandoning ordering entirely — no range query support at all. This table is worth returning to at the end of next week, once hash tables are covered in full, as the complete reference for this entire unit's structure choices.

Exercise

As an exercise, implement `ceiling(node, target)`, symmetric to this lecture's `floor` function, and trace it by hand for target 7 on Lecture 1's example tree, confirming it correctly returns 8 (the smallest

stored value greater than or equal to 7). Separately, implement `treesort` using a plain, non-AVL BST, and empirically measure its running time on an already-sorted input of size 1,000 versus a randomly-ordered input of the same size, confirming the dramatic O(n²)-versus- O(n log n) difference this lecture predicted for the two cases.

Revisiting Week 6's leaderboard case study, resolved

Week 6 Lecture 4's leaderboard case study explicitly needed a "maintained ranking structure" to support live score updates and cheap "top 10" reads, without yet having the tools to specify one concretely. A balanced BST, keyed by player score, is now a complete and specific answer: score updates cost $O(\log n)$ (delete the old value, insert the new one), and "top 10" queries are answered by a bounded traversal starting from the tree's maximum value and walking backward through 10 elements — a direct, concrete resolution of the gap that case study deliberately left open, demonstrating that sorting algorithms and tree structures are complementary tools applied to the same underlying system design problem.

Quiz — choose the structure

Given a system that needs to store employee salaries, supporting both fast lookup by employee ID and frequent range queries such as "show me everyone earning between $50,000 and $70,000," determine the most appropriate structure — a plain array, a hash table, or a balanced BST — before checking the answer.

Quiz — answer

A balanced BST, keyed by salary, is the correct choice: the range query is answered directly by this lecture's `range_query` function, in $O(\log n + k)$. A hash table, covered next week, would offer excellent lookup performance but cannot efficiently answer the range query at all, since it maintains no ordering among keys — answering the range query would require scanning all n entries regardless. A plain unsorted array fails both requirements outright, and even a sorted array, while capable of answering range queries via binary search, would cost O(n) per insertion to maintain sorted order, exactly Week 2 Lecture 4's established cost for inserting into a sorted array.

Summary

Range, floor, and ceiling queries all exploit the BST property to prune entire subtrees that cannot contain a relevant result, costing $O(\log n)$ or $O(\log n + k)$ rather than a full linear scan. Treesort is a genuine, correct sorting algorithm falling directly out of combining this week's insertion and traversal operations, though rarely used in practice due to constant-factor and memory overhead compared to

Week 6's dedicated sorting algorithms. A balanced BST provides a complete ordered-set abstraction — insertion, deletion, search, range queries, and floor/ceiling queries, all in $O(\log n)$ — resolving exactly the "maintained structure" gap Week 6 Lecture 4's leaderboard case study identified. Augmented trees extend this further to order-statistic queries through incrementally-maintained per-node metadata, a general and reusable technique. This closes Week 7's coverage of binary trees. Next week introduces hash tables and heaps, which trade away the tree's ordering guarantee in exchange for even faster average-case operation costs under the right circumstances.